Rust



Lukáš Hozda

2025-10-18

Braiins Systems s.r.o

Organizational stuff

Organizational stuff



- This semester - basic Rust
- Next semester - advanced Rust
- The subject
  - 13 lessons from 2.10.2025
  - Odd lessons - lectures
  - Even lessons - labs
    - Ideally bring your laptop, so you can follow along :)
- Homework
  - ReCodEx
  - 60% to pass
- We want this subject to be open
  - Bring friends and whoever else may be interested :)
  - Feel free to share course materials

Contacts and getting help



- I am not using MS Teams lol
- Discord server:
  - https://discord.gg/E4XpeWbJpx
- Rust CZ/SK discord:
  - https://discord.gg/ZPdaXex8
- You can reach out to me directly as well



Meet your torturers

- Lukas Hozda
- Dawid Kubis
- Pavel Simerda

Rust background and history



- Main inspirations:
  - C++: Performance, zero-cost abstractions
  - Haskell: Functional programming concepts, type system
  - Cyclone: Memory safety without garbage collection
  - Erlang: Concurrency model
  - OCaml: Type inference, pattern matching, syntax inspiration
- History of Rust:
  - 2006: Personal project by Mozilla employee Graydon Hoare
  - 2009: Mozilla begins sponsoring the project
  - 2010: Rust announced to the public
  - 2015: Rust 1.0 released
  - 2018: Non-Lexical Lifetimes (NLL) stabilized
  - 2019: Async/await stabilized
  - 2021: Rust Foundation established

Why Rust?



- Memory safety without garbage collection
- Concurrency without data races
- Zero-cost abstractions
- Modern language features (pattern matching, closures, etc.)
- Excellent package manager (Cargo)
- Strong type system and ownership model
- Cross-platform support
- Growing ecosystem and community



Programming Socks: Is it high time for thigh-highs? An investigation into the perceived unreasonable effectiveness of Programming Socks on productivity levels in the field of Software Engineering

Ian F.V.G. Hunter

Unaffiliated/Disowned, Ireland

Abstract

Programming Socks are an essential piece of equipment for any respectable software engineer. However, there is limited research on the level of impact that they have on practitioners. In this paper we investigate the claim that Programming Socks are a low-cost high-impact upgrade to engineers' wardrobes and find that they have much untapped potential.

1 Introduction

Programming Socks are thigh length or "Thigh-High" socks, commonly striped but not as a critical necessity (See Figure 1 for an example). They have been long-hailed as essential items to improve programmer performance, code quality as well as job satisfaction.

Installing Rust



- Use Rustup: https://rustup.rs/
  - Manages Rust versions and toolchains
  - Easy to update and switch between versions
- Discouraged: Using package managers (apt, brew, etc.)
- Windows: Use Rustup installer
  - Consider Windows Subsystem for Linux (WSL) for better experience
- macOS: Use Rustup via Terminal
- Linux: Use Rustup via Terminal (preferred method)

Basic setup



- Ensure Rust Analyzer is installed:
  - VS Code: Install “rust-analyzer” extension
  - Other editors: Follow editor-specific instructions
- Install rustfmt and rust-src:
  - Usually comes with Rustup installation
  - If not:

```
rustup component add rustfmt
rustup component add rust-src
```

- Verify installation:
  - `rustc --version`
  - `cargo --version`
  - `rustfmt --version`

VS Code/VS Codium



- Probably easiest option with best support
- Install “rust-analyzer” extension
  - Provides intelligent code completion and navigation
- Rust works well with debuggers
  - Use “CodeLLDB” extension for debugging Rust code
- Additional useful extensions:
  - “Better TOML” for Cargo.toml editing
  - “crates” for dependency version management

Other editors



- Neovim or Emacs is recommended
- RustRover is also good
  - https://www.jetbrains.com/rust/

Dependencies



- Rust's standard library (std) is deliberately small
  - Focuses on core functionality
- Foreign dependencies are often needed
- Finding and getting dependencies:
  - crates.io: Official Rust package registry
  - libs.rs: Better UI for crates.io with recommendatiosn
  - awesome-rust: Curated list of Rust libraries and resources
- In Rust parlance — crates == packages
  - Both library and binary
- Use Cargo to manage dependencies:
  - Add to Cargo.toml
  - `cargo add <crate-name>` to automatically add and fetch

What makes Rust different?



- Ownership and borrowing:
  - Unique approach to memory management
  - Eliminates entire classes of bugs at compile-time
- Traits instead of traditional OOP:
  - No inheritance, favors composition
  - Interfaces through traits for polymorphism
- Pattern matching:
  - Powerful and expressive way to destructure data
- Enums with associated data:
  - More expressive than enums in other languages
- Macros:
  - Powerful metaprogramming capabilities
- Zero-cost abstractions:

What makes Rust different?



- High-level constructs compile to efficient low-level code

- Fearless concurrency:
  - Type system prevents data races

- No null or exceptions:
  - Uses Option<T> for nullable values
  - Result<T, E> for error handling

- Lifetimes:
  - Explicit control over reference validity

- Cargo ecosystem:
  - Integrated build system and package manager

Hello world



```rust
fn main() {
    println!("Hello, World!");
}
```

- Save as **hello.rs**
- Compile: `rustc hello.rs`
- Run: `./hello`

Congratulations, you will never use **rustc** directly again through your entire career

Primitive Types



```rust
fn main() {
    // Ints
    let x: i32 = 42; let y: u64 = 1337;
    // Floats
    let pi: f64 = 3.14159;
    // Bools
    let am_i_locked_in: bool = true;
    // Chars
    let letter: char = 'A';
    // String slices
    let greeting: &str = "Hello, Rust!";
    // Printing values
    println!("x = {}, y = {}", x, y);
    println!("pi = {:.2}", pi);
    println!("Letter: {}", letter);
    println!("{}", greeting);
}
```



Type	Size	Description
i8, i16, i32, i64, i128	8 to 128 bits	Signed integers
u8, u16, u32, u64, u128	8 to 128 bits	Unsigned integers
isize, usize	arch-dependent	Pointer-sized integers
f32, f64	32 or 64 bits	Floating-point numbers
bool	1 byte	Boolean (true or false)
char	4 bytes	Unicode scalar value
()	0 bytes	Unit type (empty tuple)

Till next



- Join discord (optional)
- Set up Rust on your machine
- Q&A?